

ICAPS Experiment Plan for boot_cold

Prepared 2026-07-10. Companion to `BOOT_COLD_PAPER.md` (the pilot writeup); this document plans the extension from pilot to an ICAPS-submittable experimental package, framed as **dynamic scheduling / replanning / anytime optimization**, not machine learning.

1. Current implemented methods

All live in `phase0/harness.py` unless noted.

method	function	mechanism
<code>cpsat_cold</code>	<code>cpsat_default_solve(prev_solution=None)</code>	one full-budget CP-SAT solve, no reuse
<code>cpsat_warm</code>	<code>cpsat_default_solve(prev_solution=prev)</code>	full-budget solve, previous solution passed via <code>add_hint</code>
<code>boot_cold</code>	<code>warm_bootstrap_solve(use_hint=False)</code>	greedy repair floor (§ below) kept as anytime pocket under an unhinted continuation
<code>boot_warm</code>	<code>warm_bootstrap_solve(use_hint=True)</code>	same floor, continuation hinted with the floor
<code>list_schedule_bootstrap</code>	(job-shop repair function)	~1ms pure-Python greedy list-scheduling repair, feasible by construction
<code>knapsack_bootstrap</code>	<code>phase0/run_knapsack_test.py</code>	knapsack analogue (drop worst value/weight until fits, greedily top up) — separate domain, not part of the JSSP CSV schema

Exploratory, not part of the boot_cold/ICAPS story (kept, not touched): `Ins_solve` , `stall_interleaved_solve` , `select_destroy_set` + 27 destroy arms in `harness.py` , and the online-learning policies in `policies.py` (`FixedArmPolicy` , `RoundRobinPolicy` , `ContextGatedSelector` , `EpsilonGreedyPolicy`) plus `reopt/` (separate top-level, git-excluded package: hardened-stream gate experiments, all of which underperformed `boot_cold` — see `BOOT_COLD_PAPER.md` §7 for the summary). These represent three independent failed attempts to *beat* boot_cold and are retained as evidence, not as candidate ICAPS baselines.

2. Current stream generator capabilities

`phase0/streams.py` : `StreamConfig` + `generate_stream` . Dataclasses `Operation` (`op_id`, `machine`, `duration`), `Job` (`job_id`, `ops tuple` — **all frozen**), `Instance` (`index`, `num_machines`, `jobs`, `outages`, `touched_ops`, `delta_kind` — **frozen**).

Deltas (`_apply_delta` , weighted random choice): `arrival` (add one job), `cancellation` (drop one job, floor of 2 jobs remaining), `duration_jitter` (one job's durations $\times$ `uniform(0.6, 1.6)`), `outage` (one machine unavailable for a random window, merged with existing outages to keep `no_overlap` feasible). No severity parameter — magnitude is a fixed distribution per delta. No due dates, weights, or release dates on `Job` . No partial-freeze support. Deterministic under seed (tested).

3. Current metrics

`phase0/metrics.py` : `primal_integral(trajecotory, best_known, budget)` (area under relative-gap-vs-time, `gap=1` before first solution) and `final_gap(result, best_known)` . No stability metrics, no timing decomposition (bootstrap vs. solver), no optimality-proof flag exposed at the metrics layer (it exists on `SolveResult` / `KResult` per-runner but isn't uniformly surfaced).

4. Current result CSV format

Not unified — every runner defines its own columns ad hoc:

- `run_stall_test.py` : `seed`, `method`, `instance`, `delta_kind`, `objective`, `best_known`, `final_gap`, `primal_integral`, `rounds`, `improving_rounds`, `initial_objective` .
- `run_knapsack_test.py` : `seed`, `method`, `instance`, `delta_kind`, `value`, `initial_value`, `proven_optimal`, `best_known`, `final_gap`, `primal_integral` (different key metric name — maximization).
- `run_budget_sweep.py` , `run_phase0.py` : their own LNS-focused schemas (`arm`, `split`, `rounds`, etc.) — irrelevant to the ICAPS JSSP story.

None currently record: bootstrap time separately from solver time, stability/move metrics, worker count, budget, instance size, or metadata (git hash, OR-Tools version, hostname, timestamp).

5. Missing pieces for ICAPS (this plan's scope)

1. Severity-scaled deltas + six new delta kinds.
2. Stability metrics (schedule-change / anytime-threshold metrics).
3. A unified, wide CSV schema (§14 of the task spec) shared by all new runners, without breaking the three existing runners' own schemas.
4. A baseline menu wide enough to isolate *why* `boot_cold` works: pure floor (`repair_only`), floor-without-pocket ablation (`repair_plus_solver_no_floor`), generic feasibility without reuse (`greedy_from_scratch` , 5 dispatch rules), reuse-but-different-mechanism (`fix_and_optimize` , `lns_prev_solution` , `local_branching_prev` , `prev_raw` , `micro_repair_cp`).
5. Arrival-specific bootstrap variants (the known weak case).
6. A preset-driven suite runner with resumability and failure isolation.
7. A second domain (RCPSP) for cross-domain transfer evidence, and static benchmark loading, at minimal-but-real scope.
8. Statistical analysis + report generation with graceful degradation (no `scipy/matplotlib` installed in this environment — confirmed by direct check: `scipy` and `matplotlib` are **not installed**; sign-test and bootstrap CI have no third-party dependency and will be used as the primary significance tooling. Wilcoxon and plots are skipped with a clear note in the report rather than failing.).

6. Exact implementation plan

Follows the task's own Execution Order (§17), and this session executes through **Step 9** (smoke + pilot run + report). Steps producing the `main` , `ablation` , `severity` , `workers` , `budgets` , `arrivals` presets are built and validated via `--dry-run` but **not executed** in this session — the task's own instruction is that these come “only after smoke/pilot are correct,” and the compute cost of the full grids (seeds 21-50 × 4 sizes × budgets up to 30s × workers up to 16 for `main` alone) is on the order of many hours to days, appropriate for an unattended multi-day run, not this session.

1. `phase0/metrics.py` : add stability metric functions (pure functions on two solutions + instance, no dependency on `SolveResult` internals so they work for any domain/format).
2. `phase0/streams.py` : add severity plumbing (`Severity` literal type, `severity` field on `StreamConfig` defaulting to reproduce *exact* current numeric behavior when unset — old tests must keep passing unmodified) and six new delta functions. Add `due_date` , `weight` , `release_date` fields to `Job` with defaults (`None` , `1.0` , `0`) — additive, does not break frozen-dataclass equality/hashing for existing callers that never set them. Add `frozen_ops:` `frozenset[str] = frozenset()` to `Instance` for `partial_schedule_freeze` .
3. `phase0/baselines.py` (new): all baseline solve functions, each returning the existing `SolveResult` type so they interoperate with `metrics.py` / `model_builder.validate_solution` unchanged.
4. `phase0/bootstrap_policies.py` (new): the arrival-specific bootstrap variants, selected via `--bootstrap-policy` , default `append` (= exactly today's `list_schedule_bootstrap` , byte-identical output — regression tested).
5. `phase0/benchmark_loaders.py` (new): Taillard-format + JSON fallback loaders → `Instance` / `Job` / `Operation` .
6. `phase0/rcpsp/` (new package): minimal but real second domain.
7. `phase0/run_icaps_jssp_suite.py` (new): unified runner, preset registry, unified CSV schema, resumability.
8. `phase0/analyze_icaps_results.py` (new): aggregate stats, sign-test + bootstrap CI (scipy-free), breakdown tables, Markdown report; plots and LaTeX tables via matplotlib **only if present**, else a note in the report.
9. `scripts/*.sh` , `README_ICAPS_EXPERIMENTS.md` .
10. Tests throughout, run once at the end plus incrementally.

7. Compatibility concerns identified

- `phase0/run_knapsack_test.py` **had a syntax error on disk** (corrupted docstring — stray text from an old planning document had been pasted into the `from __future__ import annotations` line, breaking the module entirely: `SyntaxError: invalid character '-'`). Fixed by restoring the clean import line; the rest of the file (verified against the version used to produce `phase0_knapsack_seeds1_2.csv`) was untouched. This is a bug fix, not a design change — noted here for the record.
- `Job / Instance / Operation` are `frozen=True` dataclasses. New fields are added with defaults at the end of each dataclass to stay additive; nothing existing sets them, so equality/hash/pickling of old instances is unaffected and all 12 existing tests continue to pass unmodified.
- `SolveResult` (in `harness.py`) is reused as-is for all new baselines rather than extended, to avoid touching the exploratory LNS code that already depends on its exact shape. Stability/timing metrics are computed at the **runner** level (from two `SolveResult`s / two solutions), not stored on the dataclass.
- No `scipy / matplotlib` in this environment. Per the task's own instruction (§18), sign test + bootstrap CI (both dependency-free) are primary; Wilcoxon and plots degrade gracefully with an explicit note rather than crashing the analysis script.
- Old runners (`run_stall_test.py`, `run_knapsack_test.py`, `run_budget_sweep.py`, `run_phase0.py`) are **not modified** beyond the syntax fix above, and their existing CSV outputs (`phase0_bootstrap_seeds1_5.csv` etc.) are **not regenerated or overwritten** by this work.